

Prefect

Modern Python Workflow Orchestration

A comprehensive study guide covering Prefect's core concepts, terminology, architecture, deployment models, state management, and real-world use cases — from simple ETL pipelines to ML training orchestration and event-driven workflows.

[Flows & Tasks](#)[Deployments](#)[Work Pools](#)[States](#)[Automations](#)[Blocks](#)[Observability](#)

Contents

1. What Is Prefect?
2. Architecture Overview
3. Core Building Blocks
 - 3.1. Flows
 - 3.2. Tasks
 - 3.3. Subflows
 - 3.4. Task Runners
4. State & Lifecycle
 - 4.1. State types and transitions
 - 4.2. Retries and timeouts
 - 4.3. Caching
5. Deployments
 - 5.1. What a deployment is
 - 5.2. flow.serve vs flow.deploy
 - 5.3. Scheduling
 - 5.4. Code storage
6. Work Pools & Workers
 - 6.1. Work pool types
 - 6.2. Work queues and priority
7. Blocks
8. Automations & Events
9. Concurrency & Rate Limits
10. Artifacts & Observability
11. Key CLI Commands
12. Typical Use Cases
13. Prefect vs. Airflow
14. Quick-Start Pattern

SECTION 1

What Is Prefect?

Prefect is an open-source Python-native workflow orchestration platform that lets you build, schedule, monitor, and recover data and ML pipelines. Unlike DAG-centric tools such as Airflow, Prefect treats ordinary Python functions as first-class workflows: adding a decorator is all it takes to gain scheduling, retries, observability, and event-driven triggers.

The project ships two tiers: Prefect OSS (self-hosted server, free) and Prefect Cloud (managed SaaS with RBAC, SSO, and additional enterprise features). Both expose the same Python API — you can develop locally and promote to Cloud without code changes.

Pillar	What it means
Python-first	Workflows are plain Python functions decorated with <code>@flow</code> / <code>@task</code> . No YAML, no DAG DSL.
Hybrid execution	Orchestration (API, UI, scheduling) is separate from execution. Your code runs in your infrastructure.
Dynamic workflows	Loops, conditionals, and runtime parameters work natively — no static DAG required.
State & recovery	Every run has a tracked state. Failed runs can resume from the last successful task.
Event-driven	Automations react to state changes, metric thresholds, or external webhooks in real time.
Observable	Structured logs, artifacts, and a live UI give full visibility into every run.

SECTION 2

Architecture Overview

Prefect separates orchestration (knowing what should run and when) from execution (actually running your code). This hybrid model means your data and secrets never leave your infrastructure — even when using Prefect Cloud.

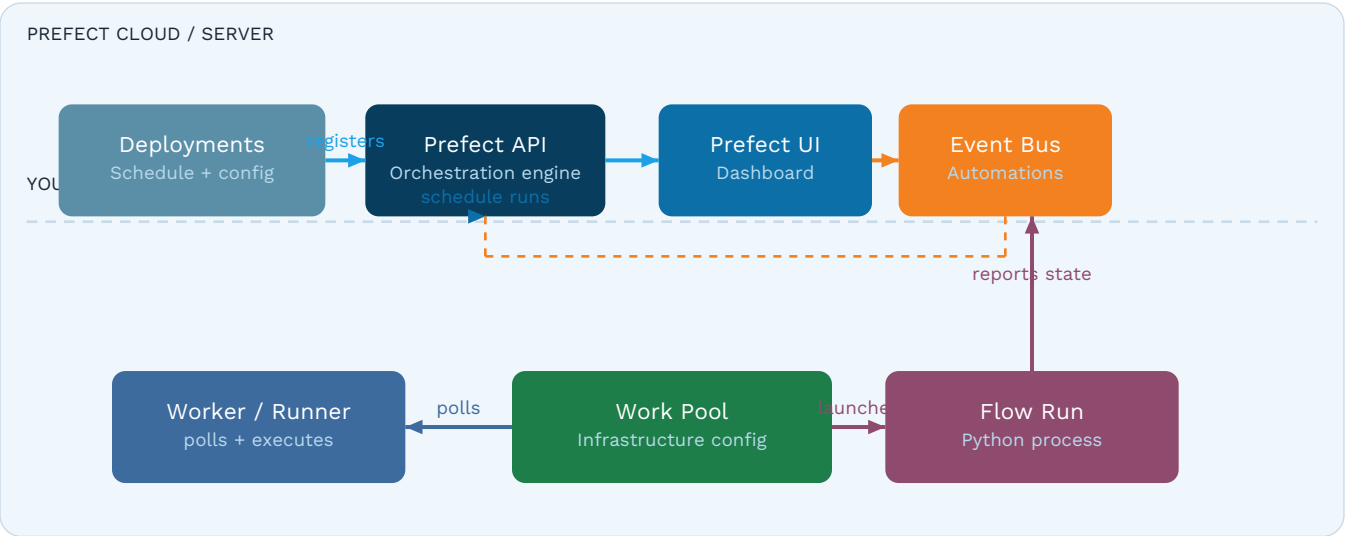


Figure 1 — Prefect architecture. The API layer lives in Cloud or a self-hosted server. Workers run in your infrastructure, polling work pools for scheduled runs. Flow code is stored separately (Git, S3, or Docker image).

The Three Planes

Plane	Components	Where it lives
Orchestration	Prefect API, UI, Event Bus, Scheduler, State DB	Prefect Cloud or self-hosted server
Execution	Workers, flow run subprocesses, task runners	Your infrastructure (K8s, ECS, local...)
Storage	Flow code, result artifacts, block credentials	Git repo, S3/GCS/Azure, Docker registry

Key Actors

Prefect Server / Cloud	Hosts the REST API, scheduler, UI, and the state database. Accepts registrations from deployments and reports from workers.
Deployment	Server-side record that describes when, where, and how a flow should run. Not the flow code itself—just the metadata.

Work Pool	A named queue that bridges the orchestration layer and your infrastructure. Defines the infrastructure type (Docker, K8s, ECS, etc.).
Worker	A lightweight long-running process in your infrastructure that polls a work pool, provisions compute, launches flow runs, and monitors them.
Flow Run	A single execution of a flow, identified by a UUID. Has a state, logs, artifacts, and parameter values.
Task Run	A single execution of a task within a flow run. Also has a state, enabling retries and caching at the task level.

SECTION 3

Core Building Blocks

3.1 Flows

A flow is any Python function decorated with `@flow`. It is the top-level unit of orchestration. When called, Prefect intercepts execution, creates a flow run record, captures logs, and tracks the final state.

Annotated `@flow`

```
from prefect import flow

@flow(name="ingest-prices",
      description="Fetch and store OHLCV data.",
      retries=2,
      retry_delay_seconds=30,
      timeout_seconds=3600,
      log_prints=True)
def ingest_prices(ticker: str = "AAPL", days: int = 90) -> dict:
    """Main entry point for price ingestion."""
    prices = fetch_prices(ticker, days) # a task
    save_to_db(prices) # another task
    return {"rows": len(prices)}
```

Parameter	Type	Effect
name	str	Display name in Prefect UI. Defaults to function name.
description	str	Markdown description. Falls back to docstring.
version	str	Optional version tag. Defaults to hash of source file.
retries	int	Times to retry on unhandled exception.
retry_delay_seconds	int / float	Wait between retries.
timeout_seconds	int	Cancel the run if it exceeds this duration.
task_runner	TaskRunner	Controls how tasks run concurrently. Default: ThreadPoolTaskRunner.
validate_parameters	bool	Enable Pydantic validation of flow parameters. Default True.
log_prints	bool	Capture print() output as Prefect log records.
flow_run_name	str / callable	Custom name for each flow run; supports {param} templates.

3.2 Tasks

A task is a Python function decorated with `@task`. Tasks are the atomic units of work — they can be retried, cached, and run concurrently. Unlike flows, tasks can only run inside a flow. Tasks have their own state (Completed, Failed, Cached...) tracked independently from the parent flow.

@task with caching and retries

```
from prefect import task
from prefect.cache_policies import INPUTS

@task(name="fetch-ohlcv",
      retries=3,
      retry_delay_seconds=10,
      cache_policy=INPUTS,          # cache by input args
      cache_expiration=timedelta(hours=1),
      timeout_seconds=120,
      tags=["external-api"])
def fetch_prices(ticker: str, days: int) -> pd.DataFrame:
    """Pull OHLCV from market data API."""
    return market_api.get(ticker, days)

# Call directly (sequential):
df = fetch_prices("AAPL", 90)

# Call via .submit() (concurrent via task runner):
future = fetch_prices.submit("AAPL", 90)
df = future.result()
```

Parameter	Effect
name	Display name. Defaults to function name.
tags	List of strings for filtering in UI and automation triggers.
retries	Retry count on unhandled exception.
retry_delay_seconds	Wait between retries (int, float, or list for exponential back-off).
timeout_seconds	Task-level timeout. Interrupts async tasks; best-effort for sync threads.
cache_policy	INPUTS, TASK_SOURCE, RUN_ID, FLOW_PARAMETERS, NO_CACHE (combinable with +).
cache_expiration	datetime.timedelta. Cached results older than this are invalidated.
log_prints	Capture print() as logs for this task only.
task_run_name	String template or callable producing the run display name.
persist_result	bool — force result persistence even when not caching.

3.3 Subflows (Child Flows)

Any `@flow`-decorated function called from within another flow creates a subflow run. Subflows are useful for grouping related tasks with their own observability context, retry logic, and independent state. They appear as nested entries in the Prefect UI Runs dashboard.

Subflow pattern

```
@flow
def train_model(dataset: str) -> str:
    """Returns path to trained model artifact."""
    df = load_data(dataset)      # task
    X, y = featurize(df)        # task
    model_path = fit_and_save(X, y) # task
    return model_path

@flow(retries=1)
def ml_pipeline():
    # train_model runs as a subflow - has its own state and logs
    path_a = train_model("train.csv")
    path_b = train_model("val.csv") # second subflow run
    evaluate(path_a, path_b)        # task
```

3.4 Task Runners

The task runner controls how tasks submitted with `.submit()` execute. It is passed to the flow-level `@flow` decorator and applies to all tasks submitted within that flow.

Task Runner	Module	Concurrency Model	Best For
ThreadPoolTaskRunner	prefect.task_runners	Threads (GIL)	I/O-bound tasks, default
ProcessPoolTaskRunner	prefect.task_runners	OS processes	CPU-bound; timeouts reliable
DaskTaskRunner	prefect_dask	Dask cluster	Large-scale data, NumPy/Pandas
RayTaskRunner	prefect_ray	Ray cluster	ML training, model serving
ConcurrentTaskRunner	prefect.task_runners	Threads (alias)	Parallel I/O
SequentialTaskRunner	prefect.task_runners	Sequential	Debugging/simple scripts

DaskTaskRunner

```
from prefect import flow
from prefect_dask import DaskTaskRunner

@flow(task_runner=DaskTaskRunner(cluster_kwargs={"n_workers": 4}))
def distributed_etl():
    futures = [transform_chunk.submit(i) for i in range(100)]
    results = [f.result() for f in futures]
```

SECTION 4

State & Lifecycle

Every flow run and task run has a state — a rich object that records what happened, when, and why. States drive all of Prefect's orchestration logic: retries, caching, automation triggers, and UI visualization.

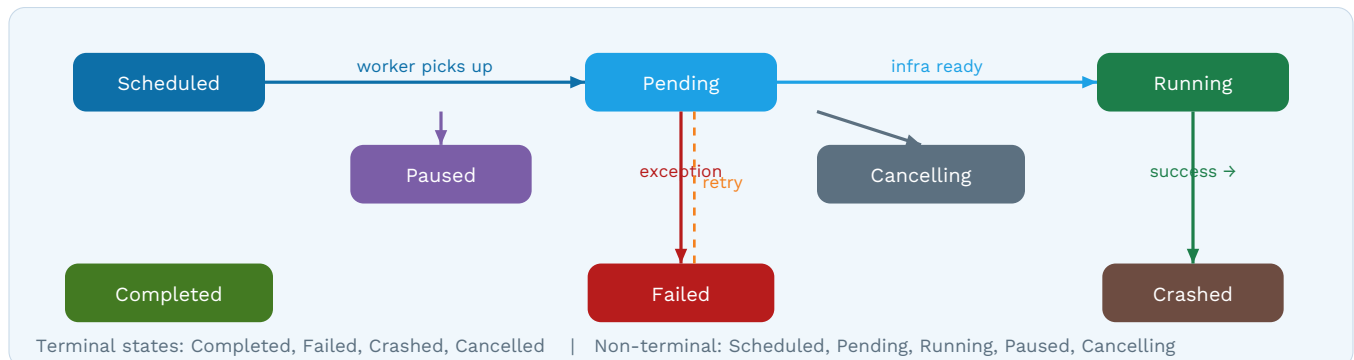


Figure 2 — Prefect state machine. State types (SCHEDULED, RUNNING, etc.) drive logic; state names (Retrying, Late, etc.) provide UI bookkeeping.

4.1 State Types

State Name	Type	Terminal ?	Typical Trigger
Scheduled	SCHEDULED	No	Manual trigger, cron schedule, or automation
Late	SCHEDULED	No	No worker picked up the run within ~15 s of scheduled time
AwaitingRetry	SCHEDULED	No	Task/flow failed but retries remain
Pending	PENDING	No	Worker picked up the run; infra being provisioned
Running	RUNNING	No	Code actively executing
Retrying	RUNNING	No	Executing again after a prior failure
Paused	PAUSED	No	Flow called <code>flow.pause_run()</code> ; awaiting manual resume
Cancelling	CANCELLING	No	User triggered cancel; infrastructure being cleaned up
Completed	COMPLETED	Yes	Returned without raising an exception
Cached	COMPLETED	Yes	Cache key matched; result loaded from prior run
Failed	FAILED	Yes	Unhandled exception raised; no retries remain

TimedOut	FAILED	Yes	Exceeded timeout_seconds
Crashed	CRASHED	Yes	OS signal (OOM, SIGKILL, Ctrl-C), not a Python exception
Cancelled	CANCELLED	Yes	User-initiated cancellation completed

4.2 Retries & Timeouts

Retries are configured at the flow or task level. Pass a list to `retry_delay_seconds` for exponential back-off:

Retries and timeouts

```
# Exponential back-off: 1s, 10s, 100s
@task(retries=3, retry_delay_seconds=[1, 10, 100])
def flaky_api_call(): ...

# Timeout — interrupts async tasks at await points; best-effort for threads
@task(timeout_seconds=30)
async def async_fetch(): ...

# Use ProcessPoolTaskRunner when you need reliable sync timeouts
@flow(task_runner=ProcessPoolTaskRunner())
def reliable_timeout_flow():
    slow_sync_task.submit() # timeout CAN interrupt process
```

4.3 Task Caching

Caching stores a task's result and returns it immediately on subsequent runs when the cache key matches — skipping re-execution entirely. The `cache_policy` parameter selects the key generation strategy:

Cache Policy	Key is derived from...	Use when...
INPUTS	Task function arguments (hashed)	Pure functions: same inputs → same output
TASK_SOURCE	Source code of the task function	Source rarely changes; output is deterministic
RUN_ID	Current flow run ID	Idempotency within a single run only
FLOW_PARAMETERS	Parent flow's parameters	All tasks share same cache scope per run
NO_CACHE	N/A — caching disabled	Non-deterministic tasks (fetch live data, etc.)

Cache policies

```
from prefect import task
from prefect.cache_policies import INPUTS, TASK_SOURCE
from datetime import timedelta

# Cache for 24 h based on function arguments
@task(cache_policy=INPUTS, cache_expiration=timedelta(hours=24))
def expensive_feature_engineering(df_hash: str) -> pd.DataFrame: ...

# Combine policies: key is hash of INPUTS + TASK_SOURCE
@task(cache_policy=INPUTS + TASK_SOURCE)
def versioned_transform(data: list) -> list: ...
```

SECTION 5

Deployments

A deployment is a server-side record that tells Prefect: when to run a flow, where to run it, and with which default parameters. Deployments are the bridge between a Python function and production scheduling.

! Deployments do not contain your flow code. They reference code stored externally (Git, S3, Docker image). This separation is fundamental to the hybrid model — Prefect Cloud never sees your code or data.

5.1 Deployment Schema (key fields)

Field	Type	Purpose
name	str	Unique identifier within the flow
entrypoint	str	path/to/flow.py:function_name
schedules	list[Schedule]	When to create new runs automatically
parameters	dict	Default parameter values (overridable per run)
work_pool_name	str	Which work pool executes this deployment
job_variables	dict	Override work pool template vars (env, image, CPU...)
tags	list[str]	Metadata for filtering, automation triggers
concurrency_limit	int	Max simultaneous active runs for this deployment
paused	bool	When True, no new runs are created from schedules
version	str	Deployment version; auto-incremented by Prefect Cloud
pull_steps	list[dict]	Steps to retrieve code before execution (e.g., git clone)

5.2 flow.serve vs flow.deploy

Prefect offers two registration approaches depending on your infrastructure needs:

	flow.serve()	flow.deploy()
Infrastructure	Static — serve() process IS the worker	Dynamic — work pool provisions infra per run
Worker needed?	No (serves itself)	Yes (or Push/Managed pool)
Docker required?	No	Optional — can use Git remote storage

Best for	Always-on local/VM deployments, small teams	Cloud, K8s, ECS — scale to zero
Code changes	Restart the serve process	Re-push image or commit; no restart

serve vs deploy

```
# — Minimal: flow.serve() —————
# Long-running process; serves itself as its own worker
if __name__ == '__main__':
    ingest_prices.serve(
        name='nightly-ingest',
        cron='0 18 * * 1-5',          # weekdays at 6 pm
        parameters={'ticker': 'SPY', 'days': 365},
    )

# — Docker + work pool: flow.deploy() —————
if __name__ == '__main__':
    ingest_prices.deploy(
        name='nightly-ingest',
        work_pool_name='ecs-prod',
        image='myregistry.io/flows:latest',
        cron='0 18 * * 1-5',
        job_variables={'cpu': '1024', 'memory': '2048'},
    )
```

5.3 Scheduling

Schedule Type	Syntax	Example
Interval	timedelta or int (seconds)	Every 30 min: timedelta(minutes=30)
Cron	Standard cron string	Weekdays at 9 am: '0 9 * * 1-5'
RRule	RFC 5545 recurrence rule	Every business day end of month, with timezone
Multiple	schedules=[...] list	Combine cron + interval on one deployment

5.4 Remote Code Storage

Rather than baking code into a Docker image, you can load it at runtime from:

Source	URI prefix	Notes
Git (public)	https://github.com/...	Specify branch/tag via entrypoint path
Git (private)	HTTPS + PAT or SSH	Use Git Credentials block for auth
S3	s3://bucket/path/	Requires prefect-aws; use AWS Credentials block
GCS	gs://bucket/path/	Requires prefect-gcp

Azure Blob	az://container/path/	Requires prefect-azure
Docker image	—	Build once; deploy many times; uses flow.deploy(image=...)

Git-backed deployment

```
# Pull code from GitHub at runtime — no Docker rebuild required
from prefect import flow

flow.from_source(
    source='https://github.com/myorg/ml-pipelines.git',
    entrypoint='pipelines/train.py:train_flow',
).deploy(
    name='train-weekly',
    work_pool_name='k8s-gpu-pool',
    cron='0 2 * * 0',    # every Sunday at 2 am
)
```

SECTION 6

Work Pools & Workers

Work pools define where flow runs execute. They bridge the Prefect orchestration layer with your compute infrastructure. Each work pool has a type (Docker, Kubernetes, ECS, etc.) and a job template that specifies default infrastructure parameters.

6.1 Work Pool Types

Type	Category	You run a worker?	Description
Process	Hybrid	Yes	Subprocess on worker host. Simple local dev.
Docker	Hybrid	Yes	Runs in a Docker container on the worker host.
Kubernetes	Hybrid	Yes	Deploys a Job/Pod in your K8s cluster.
AWS ECS	Hybrid	Yes	Runs on ECS (EC2 or Fargate).
GCP Cloud Run	Hybrid	Yes	Runs a Cloud Run job.
Azure Container Instances	Hybrid	Yes	ACI container per flow run.
ECS – Push	Push	No	Prefect pushes to Fargate; no worker needed.
Cloud Run – Push	Push	No	Prefect pushes to Cloud Run directly.
ACI – Push	Push	No	Prefect pushes to Azure Container Instances.
Modal	Push	No	Prefect pushes to Modal serverless compute.
Prefect Managed	Managed	No	Prefect provisions and manages all infra.

6.2 Work Queues and Priority

Each work pool contains one or more work queues. Queues have a numeric priority (lower = higher priority) and an optional concurrency limit. This enables fine-grained control:

- A critical queue (priority 1, concurrency 1) for urgent backfills.
- A high queue (priority 2, no limit) for scheduled production runs.
- A low queue (priority 10, concurrency 2) for exploratory ad-hoc runs.

Work pool CLI + job_variables

```
# Create a pool and start a worker
$ prefect work-pool create --type kubernetes my-k8s-pool
$ prefect worker start --pool my-k8s-pool

# Preview what will run in the next 12 hours
$ prefect work-pool preview my-k8s-pool --hours 12

# Override job variables per deployment
my_flow.deploy(
    work_pool_name='my-k8s-pool',
    job_variables={
        'image': 'myregistry.io/flows:v2',
        'env': {'MODEL_VERSION': '3.1'},
        'cpu_request': '2',
        'memory_request': '4Gi',
    }
)
```

> Workers poll every 15 seconds by default. Use `--prefetch-seconds` to allow time for infra provisioning so runs start on schedule. With Push work pools there is no worker — Prefect Cloud triggers infra directly.

SECTION 7

Blocks

A block is a Prefect-managed configuration object that persists in the Prefect backend. Blocks hold reusable, encrypted settings — database URLs, cloud credentials, Slack tokens — that can be loaded by name at runtime. They are built on Pydantic models, so fields are validated automatically.

Block (from prefect or integration package)	What it stores / does
S3Bucket	AWS S3 bucket name + AWS credentials; <code>.read_path()</code> / <code>.write_path()</code>
GCSBucket	GCS bucket + Google credentials
AzureBlobStorageCredentials	Azure storage account + connection string
AwsCredentials	AWS access key + secret; used by S3Bucket, ECS infra, etc.
GcpCredentials	GCP service account JSON; used by GCSBucket, Cloud Run, etc.
AzureCredentials	Azure service principal; used by ACI, Azure Blob, etc.
GitRepository	Git repo URL + credentials; used as deployment pull step
Secret	Encrypted arbitrary string value (API key, password, token)
SlackWebhook	Slack webhook URL; <code>.notify()</code> posts a message
MSTeamsWebhook	Teams channel webhook; <code>.notify()</code> posts a message
JSON	Arbitrary JSON blob stored as a named block
String	Arbitrary string stored as a named block
DockerRegistry	Registry URL + credentials; used by Docker infrastructure
KubernetesClusterConfig	kubeconfig content; used by Kubernetes work pools

Creating and loading a block

```
# 1. Save a block (once, in setup or CLI)
from prefect_aws import S3Bucket, AwsCredentials

creds = AwsCredentials(
    aws_access_key_id='AKID...',
    aws_secret_access_key='secret'
)
creds.save('prod-aws-creds')

S3Bucket(bucket_name='my-ml-bucket', credentials=creds).save('ml-bucket')

# 2. Load by name inside a flow — credentials never in code
from prefect_aws import S3Bucket

@task
def save_model(model_bytes: bytes):
    bucket = S3Bucket.load('ml-bucket')
    bucket.write_path('models/latest.pkl', model_bytes)
```

SECTION 8

Automations & Events

Automations make Prefect reactive. You define a trigger (a condition to watch for) and an action (what to do when it fires). Triggers are backed by a real-time event bus, so reactions happen within seconds of the condition being met.

8.1 Trigger Types

- Flow/task run state change — e.g., any run transitions to Failed.
- Work pool status — e.g., work pool becomes unhealthy (no workers polling).
- Deployment status — e.g., a deployment is paused or a run is late.
- Metric threshold — e.g., average run latency exceeds 10 min over 30 min window.
- Custom event — any event emitted via `emit_event()` from your own code or webhooks.
- Compound / sequence — require multiple events within a time window (e.g., 3 failures in 5 min).

8.2 Actions

Action	When to use
Run a deployment	Trigger a downstream flow run (e.g., reprocessing after an upstream fix)
Cancel / change run state	Kill a stuck run or force it to a custom state
Send a notification	Alert via Slack, Teams, or email on failure or SLA breach
Call a webhook	Trigger any external system (PagerDuty, GitHub, custom API)
Pause / resume a work pool	Stop processing when infra is over-budget; resume on cron
Pause / resume a deployment schedule	Freeze scheduled runs during a maintenance window
Suspend / resume a flow run	Pause execution mid-flow awaiting human approval

Custom events + automation hook

```
# Emit a custom event from within a task or flow
from prefect.events import emit_event

@task
def check_data_quality(df):
    if df.isnull().mean().max() > 0.10:
        emit_event(
            event='data.quality.alert',
            resource={'prefect.resource.id': 'dataset/prices'},
            payload={'null_rate': df.isnull().mean().max()},
        )
    return df

# Automation: when 'data.quality.alert' fires,
# → send Slack notification + pause downstream deployment schedule
```

> Jinja templates in notification actions give you full access to flow run context: `{{ flow_run.name }}`, `{{ flow.name }}`, `{{ deployment.parameters }}`, `{{ event.id }}`. Use `{{ flow_run|ui_url }}` to embed a direct link to the failed run.

SECTION 9

Concurrency & Rate Limits

Prefect provides two layers of concurrency control: deployment-level limits (max simultaneous runs of one deployment) and global concurrency limits (named slots usable by any task or flow, across deployments).

Global Concurrency Limits

Global limits are named slot pools. A task occupies one or more slots while executing and releases them on completion. Adding `slot_decay_per_second` converts them into rate limiters: slots decay at a fixed rate, making it trivial to implement API call throttling.

Global concurrency limits

```
# Create a limit: max 5 concurrent database connections
$ prefect gcl create database --limit 5

# Rate-limit: 10 API calls per second
$ prefect gcl create market-api --limit 10 --slot-decay-per-second 10

# Use in a task
from prefect.concurrency.sync import concurrency, rate_limit

@task
def query_db(sql: str):
    with concurrency('database', occupy=1): # blocks until slot free
        return db.execute(sql)

@task
def fetch_quote(ticker: str):
    rate_limit('market-api') # blocks until token available
    return market_api.quote(ticker)
```

At the deployment level, set `concurrency_limit` in the deployment schema or via `flow.deploy(concurrency_limit=N)`:

Deployment-level concurrency

```
# At most 2 simultaneous runs of this specific deployment
my_flow.serve(
    name='etl-prod',
    concurrency_limit=2,
    # collision_strategy='CANCEL_NEW' (default) or 'ENQUEUE'
)
```

SECTION 10

Artifacts & Observability

Prefect provides three observability mechanisms: structured logs (sent to the Prefect backend and visible in the UI), artifacts (persisted outputs attached to runs), and events (emitted for every state change and custom trigger).

10.1 Artifacts

Artifact Type	Function	Renders As
Link	<code>create_link_artifact(key, link, link_text, desc)</code>	Clickable URL in Artifacts tab
Markdown	<code>create_markdown_artifact(key, markdown, desc)</code>	Rendered Markdown content
Table	<code>create_table_artifact(key, table, desc)</code>	Interactive table (list of dicts)
Image	<code>create_image_artifact(key, image_url, desc)</code>	Inline image preview
Progress	<code>create_progress_artifact()</code> + <code>update_progress_artifact()</code>	Live progress bar

Artifacts in a task

```
from prefect.artifacts import create_table_artifact, create_markdown_artifact

@task
def evaluate_model(model_path: str, test_df) -> dict:
    metrics = compute_metrics(model_path, test_df)

    # Persist metrics as a named, versioned artifact
    create_table_artifact(
        key='model-metrics',
        table=[metrics],          # list of dicts
        description='# Evaluation Results',
    )

    create_markdown_artifact(
        key='eval-report',
        markdown=f'## Model: {model_path}\n**Accuracy**: {metrics["acc"]:.3f}',
    )
    return metrics
```

10.2 Logging

Prefect captures logs through three channels: the Prefect run logger (`get_run_logger()`), `print()` statements when `log_prints=True`, and standard Python logging (auto-captured for the root logger).

Structured logging

```
from prefect import flow, task
from prefect.logging import get_run_logger

@task
def process(data):
    logger = get_run_logger() # associates logs with this task run
    logger.info('Processing %d rows', len(data))
    logger.debug('First row: %s', data[0])

@flow(log_prints=True)      # print() → Prefect logs
def my_flow():
    print('Flow started')   # captured automatically
    process([1, 2, 3])

# Retrieve from CLI
# $ prefect flow-run logs <FLOW-RUN-ID> > run.log
```

SECTION 11

Key CLI Commands

Command	What it does
<code>prefect server start</code>	Start self-hosted Prefect server (SQLite by default)
<code>prefect cloud login</code>	Authenticate to Prefect Cloud
<code>prefect deployment run 'flow/dep'</code>	Trigger an immediate flow run of a deployment
<code>prefect deployment ls</code>	List all deployments
<code>prefect deployment inspect 'f/d'</code>	Show full deployment schema
<code>prefect deployment pause 'flow/dep'</code>	Pause the deployment's schedules
<code>prefect flow-run ls</code>	List recent flow runs
<code>prefect flow-run logs</code>	Stream logs for a specific flow run
<code>prefect flow-run cancel</code>	Cancel a running or scheduled flow run
<code>prefect work-pool create --type docker my-pool</code>	Create a Docker work pool
<code>prefect work-pool ls</code>	List all work pools and their status
<code>prefect worker start --pool my-pool</code>	Start a worker polling a work pool
<code>prefect gcl create db --limit 5</code>	Create a global concurrency limit (5 slots)
<code>prefect block ls</code>	List all saved blocks
<code>prefect block register -m prefect_aws</code>	Register blocks from prefect-aws
<code>prefect profile ls</code>	List configured profiles (server endpoints, API keys)
<code>prefect config set PREFECT_API_URL=...</code>	Set CLI server endpoint URL

SECTION 12

Typical Use Cases

ETL / Data Engineering

Extract from databases, APIs, or files; transform with Pandas/Polars/dbt; load to data warehouses or lakes. Retries handle transient API failures; caching avoids re-fetching unchanged data.

- Schedule nightly ingestion from REST APIs
- Incremental loads with `cache_policy=INPUTS`
- Fan-out extraction with `.submit()` + `DaskTaskRunner`
- Trigger dbt Cloud jobs via Prefect automations

ML Training Pipelines

Orchestrate end-to-end ML workflows: data prep → feature engineering → training → evaluation → model registration. Subflows isolate each stage; artifacts capture metrics and model paths.

- Parameterized training runs (hyperparameter search)
- Cache feature engineering output for 24 h
- Trigger retraining on data-quality alert events
- Log metrics to MLflow / W&B; inside `@task`

Model Serving & Inference Pipelines

Batch inference jobs, model warm-up checks, and periodic A/B evaluation. Scale with Kubernetes work pool; rate-limit prediction API calls with global concurrency limits.

- Scheduled batch scoring on S3 data
- Rate-limit calls to external LLM APIs
- Progress artifacts for long-running batch jobs
- Auto-pause inference schedule when model drift detected

Event-Driven / Reactive Workflows

Trigger flows from external events (file uploads, Kafka messages, webhooks) or from within Prefect (state changes, metric alerts). The event bus delivers triggers within seconds.

- S3 file-arrival event → kick off ETL
- Flow failure → notify Slack + trigger backfill
- Work pool unhealthy → page on-call via PagerDuty webhook
- Compound trigger: 3 failures in 5 min → escalation

Data Quality & Monitoring

Run validation checks as tasks; emit custom events when rules fail; surface results as table artifacts with lineage history. Automate remediations (pause downstream, alert, re-run with clean data).

- Great Expectations / Pandera inside @task
- `create_table_artifact` for per-column statistics
- Automation: null-rate threshold → pause ETL + notify
- Historical artifact tracking with key-based versioning

Platform / Internal Tooling

Platform teams expose opinionated work pools (guarded job templates) to data teams. CI/CD pipelines call `prefect deployment run` to promote code. Infrastructure-as-code via Terraform Prefect provider.

- Work pool job template enforces resource guardrails
- GitHub Actions step: `prefect deploy --all`
- Terraform manages work pools + concurrency limits
- Multi-tenant RBAC in Prefect Cloud workspaces

SECTION 13

Prefect vs. Apache Airflow

Airflow is the incumbent enterprise orchestration platform (2015). Prefect positions itself as the modern, Python-native alternative. Key differences:

Attribute	Prefect	Airflow
Workflow definition	Plain Python (@flow / @task)	DAG class + Operators DSL
Dynamic workflows	Native (loops, conditionals)	Limited (Dynamic DAGs in 2.x only)
Execution model	Hybrid — orchestration ≠ execution	Centralized scheduler + workers
Infrastructure overhead	Low — only workers needed	High — scheduler, web server, DB, workers
Cost (cloud)	60-70% lower vs Airflow/Astronomer	Higher due to always-on scheduler
Local development	Run flows as regular Python scripts	Requires Airflow webserver + scheduler
Retry logic	Per-task + per-flow; exponential back-off list	Per-task; linear delay
Event-driven triggers	Built-in real-time event bus	Polling via Sensors only
Observability	Artifacts, structured logs, events, UI	Logs + DAG graph UI
Community & ecosystem	Smaller but growing; modern integrations	Mature ecosystem; 1000+ providers
Best for	Dynamic/ML, Python-centric, modern data teams	Static ETL, large enterprises, heavy Airflow investment

! Use Prefect when: You value Python-native simplicity, ML/AI workloads, dynamic workflows, or want to minimize infrastructure overhead. **Consider Airflow when:** You have heavy existing Airflow investment, need 1000+ community providers, or work in a DAG-centric data engineering culture.

SECTION 14

Quick-Start Pattern

The following annotated example shows all core concepts wired together: a parameterized flow with tasks, caching, retries, an artifact, and a deployment with a cron schedule.

Annotated end-to-end example

```

# full_example.py
from prefect import flow, task
from prefect.cache_policies import INPUTS
from prefect.artifacts import create_table_artifact
from prefect.logging import get_run_logger
from datetime import timedelta
import pandas as pd

# — Tasks —
@task(retries=3, retry_delay_seconds=[5, 30, 120],
      cache_policy=INPUTS, cache_expiration=timedelta(hours=6))
def extract(source: str) -> pd.DataFrame:
    return pd.read_parquet(source)

@task
def transform(df: pd.DataFrame) -> pd.DataFrame:
    return df.dropna().assign(processed=True)

@task
def load(df: pd.DataFrame, dest: str) -> int:
    df.to_parquet(dest, index=False)
    create_table_artifact(
        key='load-summary',
        table=[{'rows': len(df), 'dest': dest}],
    )
    return len(df)

# — Flow —
@flow(name='etl-pipeline', log_prints=True)
def etl(source: str = 's3://bucket/raw/', dest: str = 's3://bucket/clean/'):
    logger = get_run_logger()
    raw = extract(source)
    clean = transform(raw)
    rows = load(clean, dest)
    logger.info('Loaded %d rows to %s', rows, dest)

# — Deployment —
if __name__ == '__main__':
    etl.serve(
        name='etl-nightly',
        cron='0 3 * * *',          # daily at 3 am
        parameters={'source': 's3://bucket/raw/', 'dest': 's3://bucket/clean/'},
    )

```

Getting started commands

```
# Installation
$ pip install prefect
$ pip install 'prefect[aws]' # add prefect-aws for S3Bucket block

# Run locally (no server needed)
$ python full_example.py # starts serve loop

# Or: run once immediately
$ python -c 'from full_example import etl; etl()'

# Start local server to use the UI
$ prefect server start
# Open http://localhost:4200

# Trigger a run via CLI
$ prefect deployment run 'etl-pipeline/etl-nightly'
```

References

Prefect Documentation — Official documentation for Prefect 3.x — concepts, how-to guides, API reference · <https://docs.prefect.io/>

Prefect Integrations — prefect-aws, prefect-gcp, prefect-azure, prefect-dask, prefect-ray and more · <https://docs.prefect.io/integrations/>

Prefect GitHub — Open-source repository; examples, issues, discussions · <https://github.com/PrefectHQ/prefect>

Prefect Discourse — Community forum for questions and best practices · <https://discourse.prefect.io/>

Prefect vs Airflow — Official comparison with benchmarks and migration guide · <https://www.prefect.io/compare/airflow>